

[Client Usage](#)[TanStack React Query \(★\)](#)[Usage](#)**Version: 11.x**

TanStack React Query

Compared to our [classic React Query Integration](#) this client is simpler and more TanStack Query-native, providing factories for common TanStack React Query interfaces like QueryKeys, QueryOptions, and MutationOptions. We think it's the future and recommend using this over the classic client, [read the announcement post](#) for more information about this change.

Quick example query

```
import { useQuery } from '@tanstack/react-query';
import { useTRPC } from './trpc';

function Users() {
  const trpc = useTRPC();

  const greetingQuery = useQuery(trpc.greeting.queryOptions({ name: 'Jerry' }));

  // greetingQuery.data === 'Hello Jerry'
}
```

Usage

The philosophy of this client is to provide thin and type-safe factories which work natively and type-safely with Tanstack React Query. This means just by following the autocompletes the client gives you, you can focus on building just with the knowledge the [TanStack React Query docs](#) provide.

```
export default function Basics() {
  const trpc = useTRPC();
  const queryClient = useQueryClient();

  // Create QueryOptions which can be passed to query hooks
  const myQueryOptions = trpc.path.to.query.queryOptions({ /** inputs */ })
  const myQuery = useQuery(myQueryOptions)
  // or:
  // useSuspenseQuery(myQueryOptions)
  // useInfiniteQuery(myQueryOptions)

  // Create MutationOptions which can be passed to useMutation
  const myMutationOptions = trpc.path.to.mutation.mutationOptions()
  const myMutation = useMutation(myMutationOptions)

  // Create a QueryKey which can be used to manipulated many methods
  // on TanStack's QueryClient in a type-safe manner
  const myQueryKey = trpc.path.to.query.queryKey()

  const invalidateMyQueryKey = () => {
    queryClient.invalidateQueries({ queryKey: myQueryKey })
  }

  return (
    // Your app here
  )
}
```

The `trpc` object is fully type-safe and will provide auto-completes for all the procedures in your `AppRouter`. At the end of the proxy, the following methods are available:

queryOptions - querying data

Available for all query procedures. Provides a type-safe wrapper around [Tanstack's queryOptions function](#). The first argument is the input for the procedure, and the second argument accepts any native Tanstack React Query options.

```
const queryOptions = trpc.path.to.query.queryOptions(
  {
    /** input */
  },
  {
    // Any Tanstack React Query options
    staleTime: 1000,
  },
);
```

You can additionally provide a `trpc` object to the `queryOptions` function to provide tRPC request options to the client.

```
const queryOptions = trpc.path.to.query.queryOptions(  
  {  
    /** input */  
  },  
  {  
    trpc: {  
      // Provide tRPC request options to the client  
      context: {  
        // see https://trpc.io/docs/client/links#managing-context  
      },  
    },  
  },  
);
```

If you want to disable a query in a type safe way, you can use `skipToken`:

```
import { skipToken } from '@tanstack/react-query';  
  
const query = useQuery(  
  trpc.user.details.queryOptions(  
    user?.id && project?.id  
    ? {  
      userId: user.id,  
      projectId: project.id,  
    }  
    : skipToken,  
    {  
      staleTime: 1000,  
    },  
  ),  
);
```

The result can be passed to `useQuery` or `useSuspenseQuery` hooks or query client methods like `fetchQuery`, `prefetchQuery`, `prefetchInfiniteQuery`, `invalidateQueries`, etc.

`infiniteQueryOptions` - querying infinite data

Available for all query procedures that takes a cursor input. Provides a type-safe wrapper around Tanstack's `infiniteQueryOptions` function. The first argument is the input for the procedure, and the second argument accepts any native Tanstack React Query options.

```
const infiniteQueryOptions = trpc.path.to.query.infiniteQueryOptions(  
  {  
    /** input */  
  },  
  {  
    // Any Tanstack React Query options  
    getNextPageParam: (lastPage, pages) => lastPage.nextCursor,  
  },  
);
```

queryKey - getting the query key and performing operations on the query client

Available for all query procedures. Allows you to access the query key in a type-safe manner.

```
const queryKey = trpc.path.to.query.queryKey();
```

Since Tanstack React Query uses fuzzy matching for query keys, you can also create a partial query key for any sub-path to match all queries belonging to a router:

```
const queryKey = trpc.router.pathKey();
```

Or even the root path to match all tRPC queries:

```
const queryKey = trpc.pathKey();
```

infiniteQueryKey - getting the infinite query key

Available for all query procedures that takes a cursor input. Allows you to access the query key for an infinite query in a type-safe manner.

```
const infiniteQueryKey = trpc.path.to.query.infiniteQueryKey({  
  /** input */  
});
```

The result can be used with query client methods like `getQueryData`, `setQueryData`, `invalidateQueries`, etc.

```
const queryClient = useQueryClient();

// Get cached data for an infinite query
const cachedData = queryClient.getQueryData(
  trpc.path.to.query.infiniteQueryKey({ cursor: 0 }),
);

// Set cached data for an infinite query
queryClient.setQueryData(
  trpc.path.to.query.infiniteQueryKey({ cursor: 0 }),
  (data) => {
    // Modify the data
    return data;
  },
);
```

queryFilter - creating query filters

Available for all query procedures. Allows creating [query filters](#) in a type-safe manner.

```
const queryFilter = trpc.path.to.query.queryFilter(
  {
    /** input */
  },
  {
    // Any Tanstack React Query filter
    predicate: (query) => {
      query.state.data;
    },
  },
);
```

Like with query keys, if you want to run a filter across a whole router you can use `pathFilter` to target any sub-path.

```
const queryFilter = trpc.path.pathFilter({
  // Any Tanstack React Query filter
  predicate: (query) => {
    query.state.data;
  },
});
```

Useful for creating filters that can be passed to client methods like `queryClient.invalidateQueries` etc.

infiniteQueryFilter - creating infinite query filters

Available for all query procedures that takes a cursor input. Allows creating [query filters](#) for infinite queries in a type-safe manner.

```
const infiniteQueryFilter = trpc.path.to.query.infiniteQueryFilter(  
  {  
    /** input */  
  },  
  {  
    // Any Tanstack React Query filter  
    predicate: (query) => {  
      query.state.data;  
    },  
  },  
);
```

Useful for creating filters that can be passed to client methods like `queryClient.invalidateQueries` etc.

```
await queryClient.invalidateQueries(  
  trpc.path.to.query.infiniteQueryFilter(  
    {},  
    {  
      predicate: (query) => {  
        // Filter logic based on query state  
        return query.state.data?.pages.length > 0;  
      },  
    },  
  ),  
);
```

`mutationOptions` - creating mutation options

Available for all mutation procedures. Provides a type-safe identify function for constructing options that can be passed to `useMutation`.

```
const mutationOptions = trpc.path.to.mutation.mutationOptions({  
  // Any Tanstack React Query options  
  onSuccess: (data) => {  
    // do something with the data  
  },  
});
```

`mutationKey` - getting the mutation key

Available for all mutation procedures. Allows you to get the mutation key in a type-safe manner.

```
const mutationKey = trpc.path.to.mutation.mutationKey();
```

subscriptionOptions - creating subscription options

TanStack does not provide a subscription hook, so we continue to expose our own abstraction here which works with a [standard tRPC subscription setup](#). Available for all subscription procedures. Provides a type-safe identify function for constructing options that can be passed to `useSubscription`. Note that you need to have either the [httpSubscriptionLink](#) or [wsLink](#) configured in your tRPC client to use subscriptions.

```
function SubscriptionExample() {
  const trpc = useTRPC();
  const subscription = useSubscription(
    trpc.path.to.subscription.subscriptionOptions(
      {
        /** input */
      },
      {
        enabled: true,
        onStart: () => {
          // do something when the subscription is started
        },
        onData: (data) => {
          // you can handle the data here
        },
        onError: (error) => {
          // you can handle the error here
        },
        onConnectionStateChange: (state) => {
          // you can handle the connection state here
        },
      },
    ),
  );

  // Or you can handle the state here
  subscription.data; // The lastly received data
  subscription.error; // The lastly received error

  /**
   * The current status of the subscription.
   * Will be one of: `idle`, `connecting`, `pending`, or `error`.
   *
   * - `idle`: subscription is disabled or ended
   * - `connecting`: trying to establish a connection
   * - `pending`: connected to the server, receiving data
   * - `error`: an error occurred and the subscription is stopped
   */
  subscription.status;

  // Reset the subscription (if you have an error etc)
  subscription.reset();

  return <>{ /* ... */ }</>;
}
```

Query Key Prefixing

When using multiple tRPC providers in a single application (e.g., connecting to different backend services), queries with the same path will collide in the cache. You can prevent this by enabling query key prefixing.

```
// Without prefixes - these would collide!  
const authQuery = useQuery(trpcAuth.list.queryOptions()); // auth service  
const billingQuery = useQuery(trpcBilling.list.queryOptions()); // billing service
```

Enable the feature flag when creating your context:

utils/trpc.ts

```
// [...]  
  
const billing = createTRPCContext<BillingRouter, { keyPrefix: true }>();  
export const BillingProvider = billing.TRPCProvider;  
export const useBilling = billing.useTRPC;  
export const createBillingClient = () =>  
  createTRPCClient<BillingRouter>({  
    links: [  
      /* ... */  
    ],  
  });  
  
const account = createTRPCContext<AccountRouter, { keyPrefix: true }>();  
export const AccountProvider = account.TRPCProvider;  
export const useAccount = account.useTRPC;  
export const createAccountClient = () =>  
  createTRPCClient<AccountRouter>({  
    links: [  
      /* ... */  
    ],  
  });
```

App.tsx

```
// [...]  
  
export function App() {  
  const [queryClient] = useState(() => new QueryClient());  
  const [billingClient] = useState(() => createBillingClient());  
  const [accountClient] = useState(() => createAccountClient());  
  
  return (  
    <QueryClientProvider client={queryClient}>  
      <BillingProvider  
        trpcClient={billingClient}  
        queryClient={queryClient}  
        keyPrefix="billing"  
      >  
        <AccountProvider  
          trpcClient={accountClient}  
          queryClient={queryClient}  
          keyPrefix="account"  
        >  
          {/* ... */}  
        </AccountProvider>  
      </BillingProvider>  
    </QueryClientProvider>  
  );  
}
```

components/MyComponent.tsx

```
// [...]  
  
export function MyComponent() {  
  const billing = useBilling();  
  const account = useAccount();  
  
  const billingList = useQuery(billing.list.queryOptions());  
  const accountList = useQuery(account.list.queryOptions());  
  
  return (  
    <div>  
      <div>Billing: {JSON.stringify(billingList.data ?? null)}</div>  
      <div>Account: {JSON.stringify(accountList.data ?? null)}</div>  
    </div>  
  );  
}
```

The query keys will be properly prefixed to avoid collisions:

```
// Example of how the query keys look with prefixes
const queryKeys = [
  [['billing'], ['list'], { type: 'query' }],
  [['account'], ['list'], { type: 'query' }],
];
```

Inferring Input and Output types

When you need to infer the input and output types for a procedure or router, there are 2 options available depending on the situation.

Infer the input and output types of a full router

```
import type { inferRouterInputs, inferRouterOutputs } from '@trpc/server';
import { AppRouter } from '../path/to/server';

export type Inputs = inferRouterInputs<AppRouter>;
export type Outputs = inferRouterOutputs<AppRouter>;
```

Infer types for a single procedure

```
import type { inferInput, inferOutput } from '@trpc/tanstack-react-query';

function Component() {
  const trpc = useTRPC();

  type Input = inferInput<typeof trpc.path.to.procedure>;
  type Output = inferOutput<typeof trpc.path.to.procedure>;
}
```

Accessing the tRPC client

If you used the [setup with React Context](#), you can access the tRPC client using the `useTRPCClient` hook.

```
import { useTRPCClient } from './trpc';

function Component() {
  const trpcClient = useTRPCClient();

  const result = await trpcClient.path.to.procedure.query({
    /** input */
  });
}
```

If you [setup without React Context](#), you can import the global client instance directly instead.

```
import { client } from './trpc';

const result = await client.path.to.procedure.query({
  /** input */
});
```